

Target SQL

The image shows two screenshots from the Google Cloud console. The top screenshot is the 'New Project' page. It features a warning banner about project quotas, a text input for 'Project name' containing 'Ankita Project 01', and a 'Location' dropdown set to 'No organization'. At the bottom are 'CREATE' and 'CANCEL' buttons. The bottom screenshot shows the 'SQL Workspace' interface. It includes an 'Explorer' sidebar on the left with a search bar and a tree view showing 'ankita-project-01'. The main area has a 'Welcome to your SQL Workspace!' message, a 'Get started' section with a 'Try the Google Trends Demo Query' card, and buttons for 'COMPOSE A NEW QUERY' and 'ADD DATA'. A checkbox at the bottom right is labeled 'Show welcome page on startup'.

console.cloud.google.com/projectcreate?previousPage=%2Fbigquery%3F_ga%3D2.266196620.2016269571.1679064005-1348959861.1679064005%26project...

Google Cloud Search (/) for resources, docs, products, and more

New Project

You have 7 projects remaining in your quota. Request an increase or delete projects. [Learn more](#)

[MANAGE QUOTAS](#)

Project name *
Ankita Project 01

Project ID: ankita-project-01. It cannot be changed later. [EDIT](#)

Location *
No organization [BROWSE](#)

Parent organization or folder

[CREATE](#) [CANCEL](#)

Activate Windows
Go to Settings to activate Windows.

SANDBOX Set up billing to upgrade to the full BigQuery experience. [Learn more](#) [DISMISS](#) [UPGRADE](#)

Explorer + ADD

Type to search

Viewing workspace resources.
[SHOW STARRED ONLY](#)

ankita-project-01 [SHOW MORE](#)

Welcome to your SQL Workspace!

Get started

Create dataset
Refresh contents

Try the Google Trends Demo Query
This simple query generates the top search terms in the US from the Google Trends public dataset.

[OPEN THIS QUERY](#) [VIEW DATASET](#)

[COMPOSE A NEW QUERY](#) [ADD DATA](#)

☒ Show welcome page on startup

Here I have created a New Project named “ankita -project 01”.



Create dataset

Project ID
ankita-project-01

CHANGE

Dataset ID *

eCommerce_Requirements

Letters, numbers, and underscores allowed

Location type ?

☒ Region
Specifying a region provides dataset colocation with other GCP services

☐ Multi-region
Letting BigQuery select a region within a group of regions provides higher quota limits

Region *

asia-south1 (Mumbai)

Default table expiration

☐ Enable table expiration ?

Default maximum table age

Days

Advanced options

▼

CREATE DATASET

CANCEL

Activate Windows

Go to Settings to activate Windows.

Here my dataset Id is “eCommerce_Requirements”.

Customer Table

console.cloud.google.com/bigquery?project=ankita-project-01&ws=11m0

Create table

Source

Create table from
Upload

Select file *
customers.csv X BROWSE ?

File format
CSV

Destination

Project *
ankita-project-01 BROWSE

Dataset *
eCommerce_Requirements

Table *
customers
Unicode letters, marks, numbers, connectors, dashes or spaces allowed.

Table type
Native table

CREATE TABLE CANCEL

Activate Windows
Go to Settings to activate Windows.

Created 8 Tables

console.cloud.google.com/bigquery?project=ankita-project-01&ws=11m0

Google Cloud Ankita Project 01 Search (/) for resources, docs, products, and more

SANDBOX Set up billing to upgrade to the full BigQuery experience. Learn more DISMISS UPGRADE

Explorer + ADD

Viewing workspace resources.
SHOW STARRED ONLY

- ankita-project-01
 - eCommerce_Requirements
 - customers
 - geolocation
 - order_items
 - order_review
 - orders
 - payments
 - products
 - sellers

SHOW MORE

Welcome to your SQL Workspace!

Get started

Try the Google Trends Demo Query
This simple query generates the top search terms in the US from the Google Trends public dataset.
OPEN THIS QUERY VIEW DATASET

+ COMPOSE A NEW QUERY + ADD DATA

☒ Show welcome page on startup

PERSONAL HISTORY PROJECT HISTORY

Activate Windows
Go to Settings to activate Windows.
REFRESH

Customer table with 10 Records

Google Cloud | Ankita Project 01 | Search (/) for resources, docs, products, and more

SANDBOX Set up billing to upgrade to the full BigQuery experience. [Learn more](#) DISMISS UPGRADE

Explorer + ADD ←

Q Type to search

Viewing workspace resources. [SHOW STARRED ONLY](#)

- ankita-project-01
 - eCommerce_Requirements
 - customers
 - geolocation
 - order_items
 - order_review
 - orders
 - payments
 - products
 - sellars

[SHOW MORE](#)

RUN SAVE SHARE SCHEDULE MORE

Only saved queries can be shared

`SELECT * FROM `ankita-ements.customers` LIMIT 10`

This query will process 8.76 MB when run.

Row	customer_id	customer_unique_id	customer_zip_code	customer_city	customer_state
1	0735e7e4298a2ebbb4664934...	fc003b1bdc0df64b4d065d9b...	59650	acu	RN
2	903b3d86e3990db01619a4eb...	46824822b15da44e983b021d...	59650	acu	RN
3	38c97666e962d4fea7fd6a83e...	b6108acc674ae5c99e29adc10...	59650	acu	RN
4	77c2f46cf580f4874c9a5751c2...	402cce5c0509000eed9e77fec...	63430	ico	CE
5	4d3ef4cfff8ad4767c199c36a...	6ba00666ab7ead5ceec279b2...	63430	ico	CE
6	3000841b86e1f9e9493b52324...	796a0b1a21f597704057184a1...	63430	ico	CE
7	3c325415ccc7e622c66dec4bc...	05d1d2d9f0161c5f397ce7fc77...	63430	ico	CE
8	04f3a7b250e3be964f01bf22bc...	c34585a0276ecc5e4fb03de75...	63430	ico	CE
9	894202b8ef01f4719a4691e79...	01a4fe5fc00bbdb0b0a4af5a53...	63430	ico	CE
10	9d715b9fb75a9d081c14126c0...	8f399f3b7ace8e6245422c9e1f...	63430	ico	CE

Geolocation with 10 records

Row	geolocation_zip	geolocation_lat	geolocation_lng	geolocation_city	geolocation_state
1	49010	-10.910514...	-37.052400...	aracaju	SE
2	49047	-10.9268145	-37.071063...	aracaju	SE
3	49030	-10.970164...	-37.061643...	aracaju	SE
4	49048	-10.940183...	-37.070850...	aracaju	SE
5	49050	-10.927157...	-37.063078...	aracaju	SE
6	49015	-10.923370...	-37.045169...	aracaju	SE
7	49045	-10.930406...	-37.067178...	aracaju	SE
8	49052	-10.922973...	-37.057752...	aracaju	SE
9	49044	-10.992080...	-37.103470...	aracaju	SE
10	49048	-10.940235...	-37.071043...	aracaju	SE

Order_items with 10 records

Row	order_id	order_item_id	product_id	seller_id	shipping_limit_date	priority
1	f09e36e258656850b92657ac5...	1	44d53f1240d6332232e4393c0...	b64d51f0435e884e8de603b16...	2018-07-09 13:31:36 UTC	
2	f9ccaff7267fd0cf076e795b1fa...	1	44d53f1240d6332232e4393c0...	b64d51f0435e884e8de603b16...	2018-08-14 14:04:44 UTC	
3	c79bdf061e22288609201ec60...	1	5304ff3fa35856a156e1170a60...	cf6f6bc4df3999b9c6440f124f...	2017-05-12 19:05:20 UTC	
4	37193e64eb9a46b7f3197762f...	1	98224bfc1eaadb3a394ec334c...	ce616e1913288884e7742faac...	2018-06-28 01:30:49 UTC	
5	95d6357ffe41aa6d2998852a7...	1	98224bfc1eaadb3a394ec334c...	ce616e1913288884e7742faac...	2018-06-12 19:15:14 UTC	
6	95d6357ffe41aa6d2998852a7...	2	98224bfc1eaadb3a394ec334c...	ce616e1913288884e7742faac...	2018-06-12 19:15:14 UTC	
7	95d6357ffe41aa6d2998852a7...	3	98224bfc1eaadb3a394ec334c...	ce616e1913288884e7742faac...	2018-06-12 19:15:14 UTC	
8	95d6357ffe41aa6d2998852a7...	4	98224bfc1eaadb3a394ec334c...	ce616e1913288884e7742faac...	2018-06-12 19:15:14 UTC	
9	95d6357ffe41aa6d2998852a7...	5	98224bfc1eaadb3a394ec334c...	ce616e1913288884e7742faac...	2018-06-12 19:15:14 UTC	
10	dde867f83e689b0167785b684...	1	914323edd50192310dd03435...	2c9e548be18521d1c43cde1c5...	2017-10-20 14:50:12 UTC	

Payments with 10 Records

Row	order_id	payment_sequence	payment_type	payment_installment	payment_value
1	1a57108394169c0b47d8f876a...	2	credit_card	0	129.94
2	744bade1fc9ff3f31d860ace07...	2	credit_card	0	58.69
3	8bcbe01d44d147f901cd31926...	4	voucher	1	0.0
4	fa65dad1b0e818e3ccc5cb0e3...	14	voucher	1	0.0
5	6ccb433e00daae1283ccc9561...	4	voucher	1	0.0
6	4637ca194b6387e2d538dc89...	1	not_defined	1	0.0
7	00b1cb0320190ca0daa2c88b3...	1	not_defined	1	0.0
8	45ed6e85398a87c253db47c2d...	3	voucher	1	0.0
9	fa65dad1b0e818e3ccc5cb0e3...	13	voucher	1	0.0
10	c8c528189310eaa44a745b8d9...	1	not_defined	1	0.0

Review with 10 Records

query 8
products
*Unsaved query 9
sellers
*Unsaved query10
*Unsaved query1

RUN
SAVE
SHARE
SCHEDULE
MORE
Query completed.

1
SELECT * FROM `ankita-` . `elements.order\_review` LIMIT 10

Query results
SAVE RESULTS
EXPLORE DATA

Row	review_id	order_id	review_score	review_comment_title	review_creation_date	review_creation_time
1	be7e2989673cb2a147b87ba73...	777c67eab7c0712ccde8ffbb2...	1	null	0001-04-17 00:00:00 UTC	00
2	e12151267e4594d69eda14a87...	4338a4463f7f9193d2a03a83a...	1	null	0001-04-17 00:00:00 UTC	00
3	41d614b133efebcd10352001b...	b8aaeda740b17cf925d3f2a13...	1	null	0001-04-17 00:00:00 UTC	00
4	c950324a42c5796d06f569f77...	b159d0ce7cd881052da94fa16...	1	null	0001-04-17 00:00:00 UTC	00
5	76823ada94c8861ecebcfbc7c...	2a3007ed051b02a0e0dd0709c...	1	null	0001-04-17 00:00:00 UTC	00
6	fe270df00abcb5c39fc7385143...	a39d3db795a5cf4c8b6c9dd05...	1	null	0001-04-17 00:00:00 UTC	00
7	1b71e0b29ec2faa0a029ded43...	0e530f6be154c9d7e7b12f341...	1	null	0001-04-17 00:00:00 UTC	00
8	efe4020a945ee6feca58606694...	264c045399fb02e9f309f715c2...	1	null	0001-04-17 00:00:00 UTC	00
9	23ee6bc2492808dca0e807e13...	2ba1366baecad3c3536f27546...	1	null	0001-04-17 00:00:00 UTC	00
10	63c6cec8cb9a2d0990b339998...	aaae80f5b6239bd9e1b22e9aa...	1	null	0001-04-17 00:00:00 UTC	00

Orders with 10 Records

Row	review_id	order_id	review_score	review_comment_title	review_creation_date	rev
1	be7e2989673cb2a147b87ba73...	777c67eab7c0712ccde8ffbb2...	1	null	0001-04-17 00:00:00 UTC	00
2	e12151267e4594d69eda14a87...	4338a4463f7f9193d2a03a83a...	1	null	0001-04-17 00:00:00 UTC	00
3	41d614b133efebcd10352001b...	b8aaeda740b17cf925d3f2a13...	1	null	0001-04-17 00:00:00 UTC	00
4	c950324a42c5796d06f569f77...	b159d0ce7cd881052da94fa16...	1	null	0001-04-17 00:00:00 UTC	00
5	76823ada94c8861ecebcfbc7c...	2a3007ed051b02a0e0dd0709c...	1	null	0001-04-17 00:00:00 UTC	00
6	fe270df00abcb5c39fc7385143...	a39d3db795a5cf4c8b6c9dd05...	1	null	0001-04-17 00:00:00 UTC	00
7	1b71e0b29ec2faa0a029ded43...	0e530f6be154c9d7e7b12f341...	1	null	0001-04-17 00:00:00 UTC	00
8	efe4020a945ee6fece58606694...	264c045399fb02e9f309f715c2...	1	null	0001-04-17 00:00:00 UTC	00
9	23ee6bc2492808dca0e807e13...	2ba1366baecad3c3536f27546...	1	null	0001-04-17 00:00:00 UTC	00
10	63c6cec8cb9a2d0990b339998...	aaae80f5b6239bd9e1b22e9aa...	1	null	0001-04-17 00:00:00 UTC	00

Products with 10 Records

Row	product_id	product_category	product_name	product_descrig	product_photos	product_weight	product_length	prc
1	5eb564652db742ff8f28759cd8...	null	null	null	null	null	null	
2	09ff539a621711667c43eba6a...	babies	60	865	3	null	null	
3	2f763ba79d9cd987b2034aac7...	electronics	45	1198	2	595	8	
4	a69f15dfb803d485e8933e80b...	Watches present	53	506	6	150	11	
5	e1cfc87f543782b8a78b59fc85...	Garden tools	39	524	4	369	26	
6	106392145fca363410d287a81...	bed table bath	58	309	1	2083	12	
7	7e33f4a1c59f89da30a335b2d...	electronics	51	381	3	1075	22	
8	bc9cc914f974963c07be697fc...	HEALTH BEAUTY	55	435	1	75	14	
9	5ae533eac9c0e93b3f89bc9ae...	computer accessories	58	1340	1	83	12	
10	67d1a56495104e195338ec90...	pet Shop	20	2153	1	275	8	

Sellers with 10 Records

< products × *Unsaved query 9 × sellers × *Unsaved query10 ×					
RUN SAVE SHARE SCHEDULE MORE					
<pre>1 SELECT * FROM `ankita-project-01.eCommerce_Requirements.sellers` LIMIT 10</pre>					
Query results SAVE RESULTS					
JOB INFORMATION RESULTS JSON EXECUTION DETAILS EXECUTION GRAPH PREVIEW					
Row	seller_id	seller_zip_code	seller_city	seller_state	
1	4be2e7f96b4fd749d52dff41f8...	69900	rio branco	AC	
2	327b89b872c14d1c0be7235ef...	69005	manaus	AM	
3	4221a7df464f1fe2955934e30f...	48602	bahia	BA	
4	651530bf5c607240ccdd89a30...	44600	ipira	BA	
5	2b402d5dc42554061f8ea98d1...	44900	irece	BA	
6	d03698c2efd04a549382afa66...	45658	ilheus	BA	
7	c72de06d72748d1a0dfb2125b...	46430	guanambi	BA	
8	fc59392d66ef99377e50356ee...	40243	salvador	BA	
9	b00af24704019bd2e1b335e70...	40130	salvador	BA	
10	eb4a59a06b3948e851a7d7a83...	41820	salvador	BA	

1. 1.1 Data type of columns in a table

- SELECT
-
-
- table_name,
- column_name,
- data_type
- FROM
- eCommerce_Requirements.INFORMATION_SCHEMA.COLUMNS
- WHERE
- table_name = 'customers'
- LIMIT 5;

Here INFORMATION_SCHEMA view contains information about field name which will give values of Table name, Column name and data types of a dataset. It retrieves metadata from the INFORMATION_SCHEMA.Columns view from the customers table in the eCommerce_Requirements dataset.

```

< nsaved query10 X *Unsaved query11 X *Unsaved query12 X *Unsaved query1 > +
RUN SAVE SHARE SCHEDULE MORE Query complete
1 #Data type of columns in a table
2 SELECT
3
4     table_name,
5     column_name,
6     data_type
7 FROM
8     eCommerce_Requirements.INFORMATION_SCHEMA.COLUMNS
9 WHERE
10    table_name = 'customers'
11    LIMIT 5;

```

Row	table_name	column_name	data_type
1	customers	customer_id	STRING
2	customers	customer_unique_id	STRING
3	customers	customer_zip_code_prefix	INT64
4	customers	customer_city	STRING
5	customers	customer_state	STRING

1. **1.2.** Time period for which the data is given

```

select min(order_purchase_timestamp)
as first_order,
max(order_purchase_timestamp)
as last_order
from `eCommerce_Requirements.orders`

```


X
 X
 X
 > +

RUN
 SAVE
 SHARE
 SCHEDULE
 MORE
 Query con

```

1 #Time period for which the data is given
2
3 select min(order_purchase_timestamp)
4       as first_order,
5       max(order_purchase_timestamp)
6       as last_order
7 from `eCommerce_Requirements.orders`
  
```

Press Alt+F1 for Accessibility

Query results
 SAVE RESULTS
 EXPLORE DATA

JOB INFORMATION
 RESULTS
 JSON
 EXECUTION DETAILS
 EXECUTION GRAPH
 PREVIEW

ⓘ This BigQuery table is being used for Change Data Capture(CDC), and this preview was last updated at Mon Apr 03 2023 17:15:43 GMT+0530 (India Standard Time). To pull real-time data, please query the table.

Row	first_order	last_order
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC

Activate Windows
 Go to Settings to activate Windows.

1.3 Cities and States of customers ordered during the given period

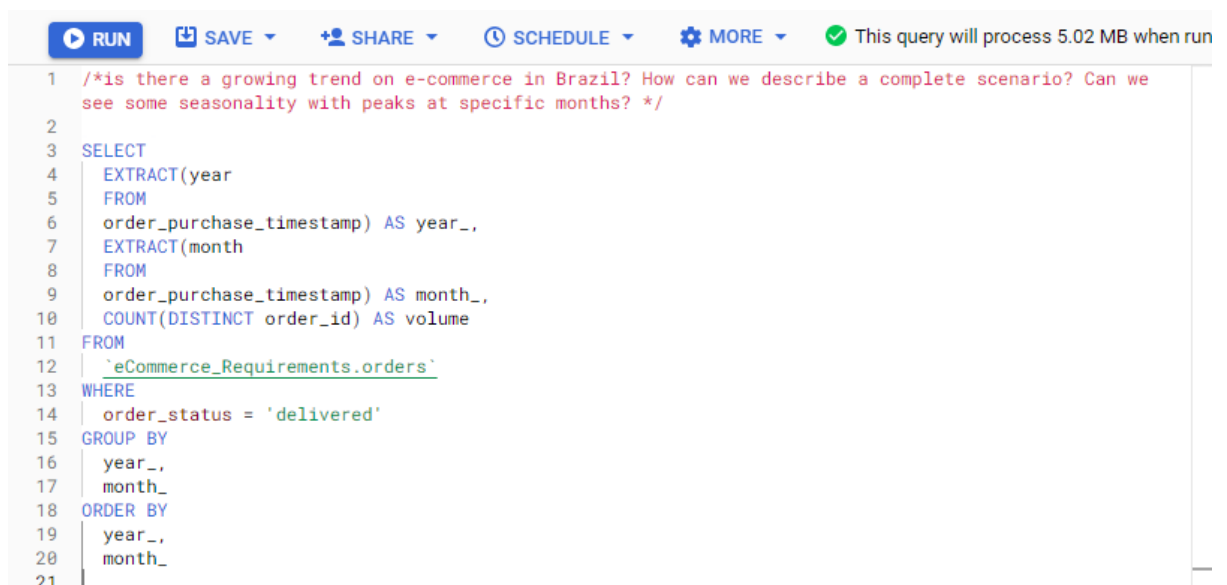
```

select DISTINCT
  cust.customer_city AS city,
  cust.customer_state AS state
FROM
  eCommerce_Requirements.customers as cust
LEFT JOIN `eCommerce_Requirements.orders` as o
ON cust.customer_id = o.customer_id
  
```

2.In-depth Exploration:

2.1 Is there a growing trend on e-commerce in Brazil? How can we describe a complete scenario? Can we see some seasonality with peaks at specific months?

```
SELECT
EXTRACT(year
FROM
order_purchase_timestamp) AS year_,
EXTRACT(month
FROM
order_purchase_timestamp) AS month_,
COUNT(DISTINCT order_id) AS volume
FROM
`eCommerce_Requirements.orders`
WHERE
order_status = 'delivered'
GROUP BY
year_,
month_
ORDER BY
year_,
month_
```



The screenshot shows a SQL query editor interface. At the top, there is a toolbar with buttons for 'RUN', 'SAVE', 'SHARE', 'SCHEDULE', and 'MORE'. A status message on the right indicates 'This query will process 5.02 MB when run'. The query is displayed in a text area with line numbers from 1 to 21. The query is a SQL statement that extracts the year and month from the 'order_purchase_timestamp' column of the 'eCommerce_Requirements.orders' table, counts the distinct 'order_id' as 'volume', and filters for 'delivered' orders. The results are grouped by year and month, and ordered by year and month.

```
1  /*is there a growing trend on e-commerce in Brazil? How can we describe a complete scenario? Can we
2  see some seasonality with peaks at specific months? */
3  SELECT
4      EXTRACT(year
5      FROM
6      order_purchase_timestamp) AS year_,
7      EXTRACT(month
8      FROM
9      order_purchase_timestamp) AS month_,
10     COUNT(DISTINCT order_id) AS volume
11  FROM
12     `eCommerce_Requirements.orders`
13  WHERE
14     order_status = 'delivered'
15  GROUP BY
16     year_,
17     month_
18  ORDER BY
19     year_,
20     month_
21
```

Row	year_	month_	volume
1	2016	9	1
2	2016	10	265
3	2016	12	1
4	2017	1	750
5	2017	2	1653
6	2017	3	2546
7	2017	4	2303
8	2017	5	3546
9	2017	6	3135
10	2017	7	2878

2.2 What time do Brazilian customers tend to buy (Dawn, Morning, Afternoon or Night)?

3. Evolution of E-commerce orders in the Brazil region:

3.1. Get month on month orders by states

```
SELECT X.MONTH , X.Year , X.Order_count , C.customer_state FROM
`eCommerce_Requirements.customers`
C
join (SELECT customer_id , extract(MONTH FROM order_purchase_timestamp) Month , extract(YEAR FROM or
der_purchase_timestamp) Year , count(order_id) Order_Count FROM
`eCommerce_Requirements.orders`
group by Year, Month , customer_id order by Year, Month) X
on X.customer_id = C.customer_id group by x.Year, x.Month ,x.Order_count , customer_state LIMIT 1000
```

```
1 /*Get month on month orders by states*/
2 SELECT X.MONTH , X.Year , X.Order_count , C.customer_state FROM
3 `eCommerce_Requirements.customers`
4 C
5 join (SELECT customer_id , extract(MONTH FROM order_purchase_timestamp) Month , extract(YEAR FROM
6 order_purchase_timestamp) Year , count(order_id) Order_Count FROM
7 `eCommerce_Requirements.orders`
8 group by Year, Month , customer_id order by Year, Month) X
9 on X.customer_id = C.customer_id group by x.Year, x.Month ,x.Order_count , customer_state LIMIT 1000
```

Row	MONTH	Year	Order_count	customer_state
1	1	2018	1	RN
2	12	2017	1	RN
3	5	2017	1	RN
4	2	2018	1	CE
5	3	2018	1	CE
6	5	2017	1	CE
7	4	2017	1	CE
8	5	2018	1	CE
9	3	2018	1	RS
10	6	2018	1	RS

3.2 Distribution of customers across the states in Brazil

SELECT

customer_state,
count(distinct customer_id)

FROM

`eCommerce\_Requirements.customers`

group by customer_state order by customer_state

```

1
2  /*Distribution of customers across the states in Brazil */
3  SELECT
4
5  customer_state,
6  count(distinct customer_id)
7
8  FROM
9  `eCommerce_Requirements.customers`
10
11 group by customer_state order by customer_state
12

```

Press Alt+F1 for Accessibility Options

Row	customer_state	f0_
1	AC	81
2	AL	413
3	AM	148
4	AP	68
5	BA	3380
6	CE	1336
7	DF	2140
8	ES	2033
9	GO	2020
10	MA	747
11	MG	11635

```

select
  cust.customer_state,
  Round(sum(oi.price),2) as sum_prices,
  round(avg(oi.price),2) as avg_prices,
  round(sum(oi.freight_value),2) as sum_frieght,
  round(avg(oi.freight_value),2) as avg_frieght,
  round(sum(oi.price+oi.freight_value),2) as price_charges,
  round(Avg(oi.price+oi.freight_value),2) as avg_price_charges

```

4.Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.

4.1Get % increase in cost of orders from 2017 to 2018 (include months between Jan to Aug only) - You can use “payment_value” column in payments table

4.2Mean & Sum of price and freight value by customer state

```

FROM
  `eCommerce_Requirements.customers` as cust
join `eCommerce_Requirements.orders` as o ON cust.customer_id = o.customer_id
join `eCommerce_Requirements.order_items` as oi ON o.order_id = oi.order_id

```

Group BY cust.customer_state order by cust.customer_state

```

1  /*Mean & Sum of price and freight value by customer state */
2  select
3      cust.customer_state,
4      Round(sum(oi.price),2) as sum_prices,
5      round(avg(oi.price),2) as avg_prices,
6      round(sum(oi.freight_value),2) as sum_frieght,
7      round(avg(oi.freight_value),2) as avg_frieght,
8      round(sum(oi.price+oi.freight_value),2) as price_charges,
9      round(Avg(oi.price+oi.freight_value),2) as avg_price_charges
10
11
12  FROM
13      `eCommerce_Requirements.customers` as cust
14      join `eCommerce_Requirements.orders` as o ON cust.customer_id = o.customer_id
15      join `eCommerce_Requirements.order_items` as oi ON o.order_id = oi.order_id
16
17  Group BY cust.customer_state order by cust.customer_state
18

```

JOB INFORMATION		RESULTS	JSON	EXECUTION DETAILS		EXECUTION GRAPH	PREVIEW
Row	customer_state	sum_prices	avg_prices	sum_frieght	avg_frieght	price_charges	avg_price_charg
1	AC	15982.95	173.73	3686.75	40.07	19669.7	213.8
2	AL	80314.81	180.89	15914.59	35.84	96229.4	216.73
3	AM	22356.84	135.5	5478.89	33.21	27835.73	168.7
4	AP	13474.3	164.32	2788.5	34.01	16262.8	198.33
5	BA	511349.99	134.6	100156.68	26.36	611506.67	160.97
6	CE	227254.71	153.76	48351.59	32.71	275606.3	186.47
7	DF	302603.94	125.77	50625.5	21.04	353229.44	146.81
8	ES	275037.31	121.91	49764.6	22.06	324801.91	143.97
9	GO	294591.95	126.27	53114.98	22.77	347706.93	149.04
10	MA	119648.22	145.2	31523.77	38.26	151171.99	183.46
11	MG	1585308.03	120.75	270853.46	20.63	1856161.49	141.38

5. Analysis on sales, freight and delivery time

5.1 Calculate days between purchasing, delivering and estimated delivery

/*Calculate days between purchasing, delivering and estimated delivery*/

select

date_diff(max(date(order_purchase_timestamp)), min(date(order_purchase_timestamp)), DAY) as purchasing,

```

date_diff(max(date(order_delivered_customer_date)), min(date(order_delivered_customer_date)), DAY) as delivering,
date_diff(max(date(order_estimated_delivery_date)), min(date(order_estimated_delivery_date)), DAY) as estimated_del,
from `eCommerce_Requirements.orders`

```

The screenshot shows a SQL query in a web-based editor. The query is as follows:

```

1 /*Calculate days between purchasing, delivering and estimated delivery*/
2
3 select
4
5 date_diff(max(date(order_purchase_timestamp)), min(date(order_purchase_timestamp)), DAY) as purchasing,
6 date_diff(max(date(order_delivered_customer_date)), min(date(order_delivered_customer_date)), DAY) as delivering,
7 date_diff(max(date(order_estimated_delivery_date)), min(date(order_estimated_delivery_date)), DAY) as
8 estimated_del,
9 from `eCommerce_Requirements.orders`

```

Below the query editor, the 'Query results' section is displayed. It includes tabs for 'JOB INFORMATION', 'RESULTS', 'JSON', 'EXECUTION DETAILS', 'EXECUTION GRAPH', and 'PREVIEW'. The 'RESULTS' tab is active, showing a table with the following data:

Row	purchasing	delivering	estimated_del
1	773	736	773

5.2 Find time_to_delivery & diff_estimated_delivery. Formula for the same given below:

- **time_to_delivery = order_purchase_timestamp - order_delivered_customer_date**
- **diff_estimated_delivery = order_estimated_delivery_date - order_delivered_customer_date**

SELECT

```

date_diff(order_purchase_timestamp, order_delivered_customer_date, DAY) as time_to_delivery,
date_diff(order_estimated_delivery_date, order_delivered_customer_date, DAY) as diff_estimated_delivery

from `eCommerce_Requirements.orders`

```


Query Editor Interface:

- Query Name: *Unsaved query2
- Status: This query will process 16.82 MB when run.
- Buttons: RUN, SAVE, SHARE, SCHEDULE, MORE

```

1 /*Group data by state, take mean of freight_value, time_to_delivery, diff_estimated_delivery */
2 select count(e1.order_id) as count_orders,
3 avg(freight_value) as avg_freight,
4 avg(date_diff(order_purchase_timestamp,order_delivered_customer_date,DAY)) as time_to_delivery ,
5 avg(date_diff(order_estimated_delivery_date,order_delivered_customer_date,DAY)) as diff_estimated_delivery
6 from
7 `eCommerce_Requirements.orders`
8 as e1 join
9 `eCommerce_Requirements.order_items`
10 as e2 on e1.order_id=e2.order_id
11 join `eCommerce_Requirements.customers`
12 as c on e1.customer_id = c.customer_id
13 where order_delivered_customer_date is not null
14 group by c.customer_state
15

```

JOB INFORMATION		RESULTS	JSON	EXECUTION DETAILS	EXECUTION GRAPH	PREVIEW
Row	count_orders	avg_freight	time_to_delivery	diff_estimated_c		
1	14146	20.9097843...	-14.689382...	11.1444931...		
2	12917	20.6258372...	-11.515522...	12.3971510...		
3	4098	21.5066276...	-14.520985...	10.6688628...		
4	46443	15.1149940...	-8.2596085...	10.2655943...		
5	2277	22.5628678...	-14.948177...	11.3728590...		
6	6133	21.6142703...	-14.708299...	13.2030001...		
7	3683	26.4875563...	-18.774640...	10.1194678...		
8	1037	27.9969141...	-17.508196...	13.6393442...		
9	375	36.5731733...	-20.978666...	9.16533333...		
10	1746	32.6933333...	-17.792096...	12.5521191...		

5.4 Sort the data to get the following:

5.5 Top 5 states with highest/lowest average freight value - sort in desc/asc limit 5

1. **SELECT**
2. `max(oi.freight_value) as Freight_max,`
3. `c.customer_state`
4. **FROM** ``eCommerce_Requirements.orders` as o inner join`
5. ``ankita-project-01.eCommerce_Requirements.order_items` as oi`
6. `on o.order_id = oi.order_id`

```

7. inner join `eCommerce_Requirements.customers` as c
8. on o.customer_id = c.customer_id
9. group by c.customer_state
10. order by c.customer_state desc
11.
12. LIMIT 5;

```

```

1 /*Sort the data to get the following:
2
3 Top 5 states with highest/lowest average freight value - sort in desc/asc limit 5 */
4
5
6
7 SELECT
8     max(oi.freight_value) as Freight_max,
9     c.customer_state
10 FROM `eCommerce_Requirements.orders` as o inner join
11     `ankita-project-01.eCommerce_Requirements.order_items` as oi
12 on o.order_id = oi.order_id
13 inner join `eCommerce_Requirements.customers` as c
14 on o.customer_id = c.customer_id
15 group by c.customer_state
16 order by c.customer_state desc
17
18 LIMIT 5;

```

Press Alt+F1 for Accessibility Options

JOB INFORMATION		RESULTS	JSON
row	Freight_max	customer_state	
1	293.27	TO	
2	339.59	SP	
3	158.38	SE	
4	375.28	SC	
5	254.55	RS	

5.6 Top 5 states with highest/lowest average time to delivery

```

SELECT
    avg(oi.freight_value) as freight_avg,
    c.customer_state
FROM `eCommerce_Requirements.orders` as o inner join
    `ankita-project-01.eCommerce_Requirements.order_items` as oi
on o.order_id = oi.order_id
inner join `eCommerce_Requirements.customers` as c
on o.customer_id = c.customer_id

group by c.customer_state
order by c.customer_state desc

```

LIMIT 5;

```
1  /*Sort the data to get the following:
2
3  Top 5 states with highest/lowest average freight value
4  - sort in desc/asc limit 5 */
5
6  SELECT
7      avg(oi.freight_value) as freight_avg,
8      c.customer_state
9  FROM `eCommerce_Requirements.orders` as o inner join
10     `ankita-project-01.eCommerce_Requirements.order_items` as oi
11     on o.order_id = oi.order_id
12     inner join `eCommerce_Requirements.customers` as c
13     on o.customer_id = c.customer_id
14     |
15     group by c.customer_state
16     order by c.customer_state desc
17
18     LIMIT 5;
```

Press Alt+F1 for Accessibility

```
1  /*Sort the data to get the following:
2
3  Top 5 states with highest/lowest average freight value
4  - sort in desc/asc limit 5 */
5
6  SELECT
7      avg(oi.freight_value) as freight_avg,
8      c.customer_state
```

Press Alt+F1 for Accessibility

Query results

[SAVE RESULTS](#)

[EXPLORE DATA](#)

JOB INFORMATION

RESULTS

JSON

EXECUTION DETAILS

EXECUTION GRAPH

PREVIEW

Row	freight_avg	customer_state
1	37.2466031...	TO
2	15.1472753...	SP
3	36.6531688...	SE
4	21.4703687...	SC
5	21.7358043...	RS

5.7 Top 5 states where delivery is really fast/ not so fast compared to estimated date

```
SELECT
    min(oi.freight_value) as freight_min,
    c.customer_state
FROM `eCommerce_Requirements.orders` as o inner join
    `ankita-project-01.eCommerce_Requirements.order_items` as oi
    on o.order_id = oi.order_id
    inner join `eCommerce_Requirements.customers` as c
    on o.customer_id = c.customer_id
where oi.freight_value > 0
group by c.customer_state
order by c.customer_state desc
```

LIMIT 5;

*2023-04-06 13:3... sc X2023-04-06 13:2... esc X2023-04-06 13:2... esc X+

RUNSAVESHARESCHEDULEMORE

```
1 SELECT
2     min(oi.freight_value) as freight_min,
3     c.customer_state
4 FROM `eCommerce_Requirements.orders` as o inner join
5     `ankita-project-01.eCommerce_Requirements.order_items` as oi
6 on o.order_id = oi.order_id
7 inner join `eCommerce_Requirements.customers` as c
8 on o.customer_id = c.customer_id
9 where oi.freight_value > 0
10 group by c.customer_state
11 order by c.customer_state desc
12
13 LIMIT 5;
```

Press Alt+F1 for Accessibility Options.

Query resultsSAVE RESULTSEXPLORE DATA

JOB INFORMATIONRESULTSJSONEXECUTION DETAILSEXECUTION GRAPHPREVIEW

Row	freight_min	customer_state
1	3.86	TO
2	0.01	SP
3	5.61	SE
4	0.04	SC
5	0.11	RS

6. Payment type analysis:

6.1 Month over Month count of orders for different payment types

/*Month over Month count of orders for different payment types */

Select

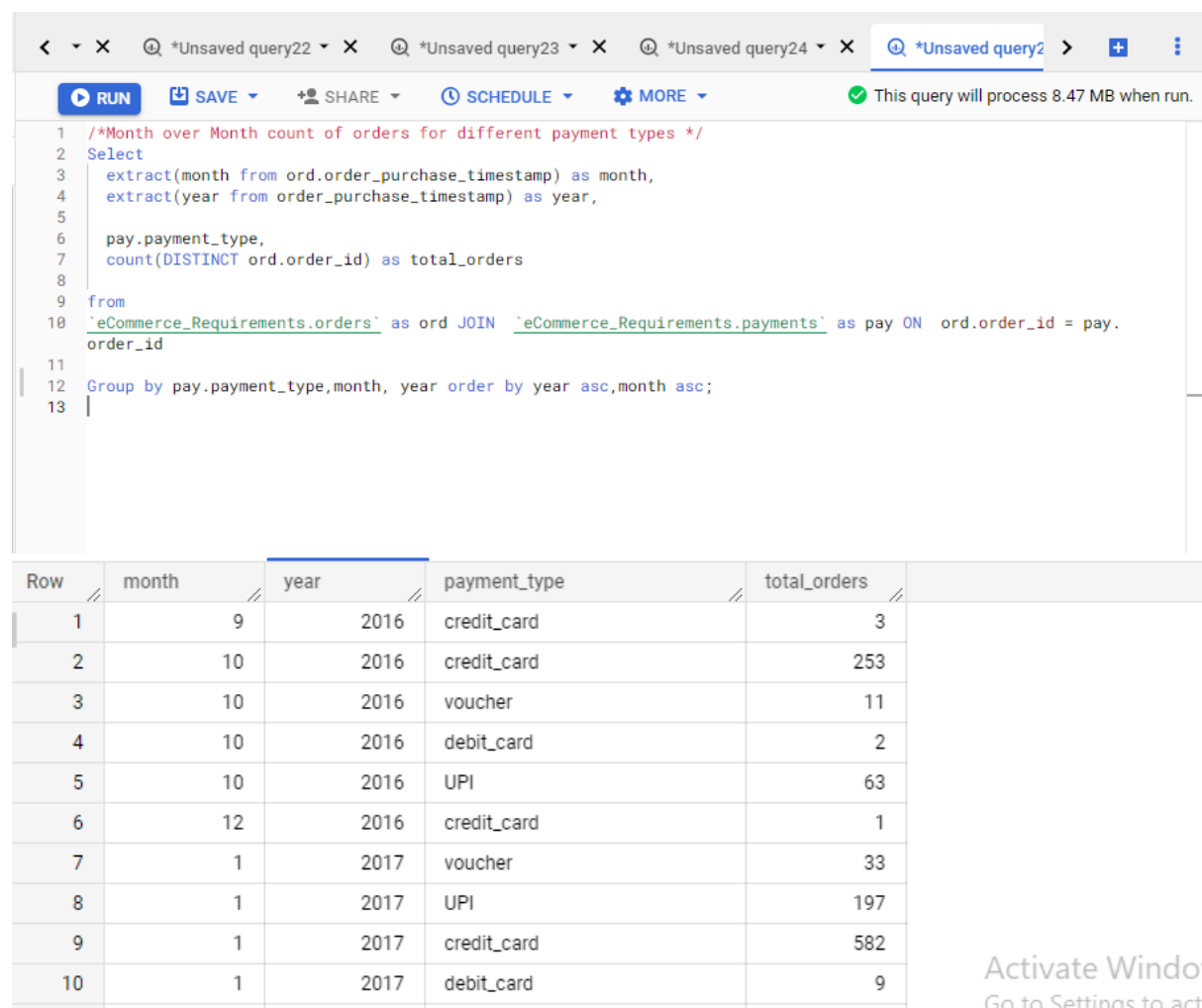
```
extract(month from ord.order_purchase_timestamp) as month,  
extract(year from order_purchase_timestamp) as year,
```

```
pay.payment_type,  
count(DISTINCT ord.order_id) as total_orders
```

from

```
`eCommerce_Requirements.orders` as ord JOIN `eCommerce_Requirements.payments` as pay ON ord.order_id = pay.order_id
```

Group by pay.payment_type,month, year order by year asc,month asc;



The screenshot shows a SQL query editor with a toolbar at the top containing buttons for RUN, SAVE, SHARE, SCHEDULE, and MORE. A status message indicates the query will process 8.47 MB. The query is as follows:

```
1 /*Month over Month count of orders for different payment types */  
2 Select  
3   extract(month from ord.order_purchase_timestamp) as month,  
4   extract(year from order_purchase_timestamp) as year,  
5  
6   pay.payment_type,  
7   count(DISTINCT ord.order_id) as total_orders  
8  
9 from  
10 `eCommerce_Requirements.orders` as ord JOIN `eCommerce_Requirements.payments` as pay ON ord.order_id = pay.  
   order_id  
11  
12 Group by pay.payment_type,month, year order by year asc,month asc;  
13
```

Below the query editor, a table displays the results of the query. The table has five columns: Row, month, year, payment_type, and total_orders. The results are as follows:

Row	month	year	payment_type	total_orders
1	9	2016	credit_card	3
2	10	2016	credit_card	253
3	10	2016	voucher	11
4	10	2016	debit_card	2
5	10	2016	UPI	63
6	12	2016	credit_card	1
7	1	2017	voucher	33
8	1	2017	UPI	197
9	1	2017	credit_card	582
10	1	2017	debit_card	9

6.2.Count of orders based on the no. of payment installments

/*Count of orders based on the no. of payment installments */

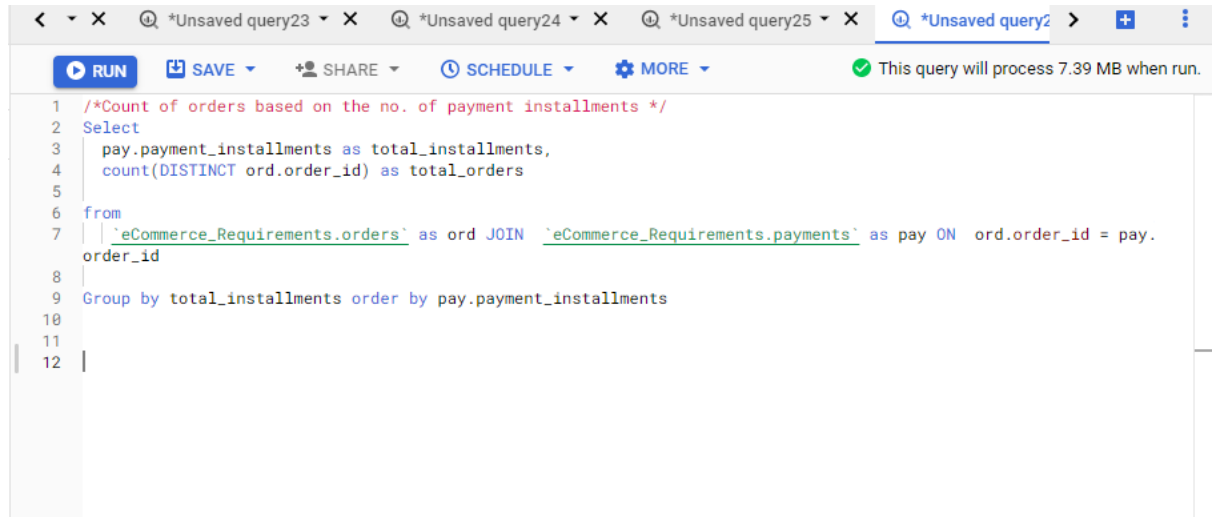
Select

```
pay.payment_installments as total_installments,  
count(DISTINCT ord.order_id) as total_orders
```

from

```
`eCommerce_Requirements.orders` as ord JOIN `eCommerce_Requirements.payments` as pay ON ord.order_id = pay.order_id
```

Group by total_installments order by pay.payment_installments



The screenshot shows a SQL query editor with a toolbar at the top containing buttons for RUN, SAVE, SHARE, SCHEDULE, and MORE. A status message on the right indicates the query will process 7.39 MB when run. The query text is as follows:

```
1 /*Count of orders based on the no. of payment installments */
2 Select
3     pay.payment_installments as total_installments,
4     count(DISTINCT ord.order_id) as total_orders
5
6 from
7     `eCommerce_Requirements.orders` as ord JOIN `eCommerce_Requirements.payments` as pay ON ord.order_id = pay.
8     order_id
9
10 Group by total_installments order by pay.payment_installments
11
12 |
```

Row	total_installment	total_orders
1	0	2
2	1	49060
3	2	12389
4	3	10443
5	4	7088
6	5	5234
7	6	3916
8	7	1623
9	8	4253
10	9	644

